Adversarial Neural Representation

A Writeup of the comma 2026 Compression Challenge

Jason Yuan
University of California, San Diego
Department of Computer Science and Engineering
jay055@ucsd.edu

May 2026

Abstract

Traditional video compression optimizes for human eyes; this challenge optimizes for neural networks. In this writeup, I present **Adversarial Neural Representation** (ANR), a novel compression pipeline that achieves a final score of 0.27 on the comma 2026 video compression challenge. Instead of wasting bits trying to reconstruct high-fidelity visual details, I framed the entire challenge as an adversarial exploit. The core idea is simple: the true "entropy" isn't the video itself, but the expected outputs of the evaluation models.

After hitting the limits of traditional codecs and realizing that on-the-fly adversarial generation violated the archive constraints, I built a specialized Implicit Neural Representation (INR), which is the core of the **Adversarial Neural Representation**. This ANR architecture features a FiLM-conditioned "Master" network with customized receptive fields specifically designed to hijack SegNet's edge detection, alongside a NeRV-style PixelShuffle "Slave" network to learn the exact residuals needed to satisfy PoseNet. To crush the model's footprint, I aggressively pruned the weights with two quantization schemes tailored for the fixed test set: Self-Compressing Networks (SCN) for the master and the entropy model, and LSQ+ int4 for the slave. Finally, to compress the underlying answer tokens without triggering the agonizing 300-hour autoregressive decoding bottleneck of standard PixelCNNs, I integrated Hierarchical Parallel Autoregressive ConvNets (HPAC), achieving a 2000× decoding speedup with an optimal bits-per-pixel trade-off. This report documents the dead ends, the mindset, and the exact architecture required to exploit the evaluation loop and compress 600 frames into pure adversarial math.

1 The reframe: entropy is the answer

When you open the comma 2026 challenge rules, you aren't looking at a traditional compression benchmark. You are looking at a highly specific optimization target:

$$\text{score} \;=\; 100 \cdot d_{\text{seg}} \;+\; \sqrt{10 \cdot d_{\text{pose}}} \;+\; 25 \cdot r. \tag{1}$$

Standard codecs like AV1 [9] or HEVC [19] didn't fail this challenge. In fact, they do a remarkably good job and set a very strong baseline. But they eventually hit an unbreakable mathematical wall. They bottom out because they optimize for the human eye. They spend precious bits preserving clouds, road textures, and lighting shifts.

The scoring function, however, doesn't ask, *"Does this look like the original video?"* It asks, *"Do the frozen SegNet and PoseNet models return the exact same numbers they would have on the original video?"*

That distinction is the key to the entire challenge. In information theory, **entropy** is the absolute theoretical lower bound of bits required to represent a piece of data without losing information. But what exactly *is* the information here?

If you are compressing for humans, the entropy is the visual state of the world. But if you are compressing for this specific scoring function, the visual state of the world contains massive amounts of noise. The evaluators don't care about the clouds. They only care about the semantic boundaries (SegNet) and the geometric camera translation (PoseNet).

Therefore, the ground-truth outputs of those two models represent the absolute lowest bound of information required to solve the equation perfectly. **The answer itself is the true entropy.** Every single bit spent on a pixel that does not actively shape those specific model outputs is a wasted bit. If I could just feed the correct SegNet labels and PoseNet vectors directly to the evaluator, I would be operating at the true entropy limit of the challenge and score a perfect zero on distortion.

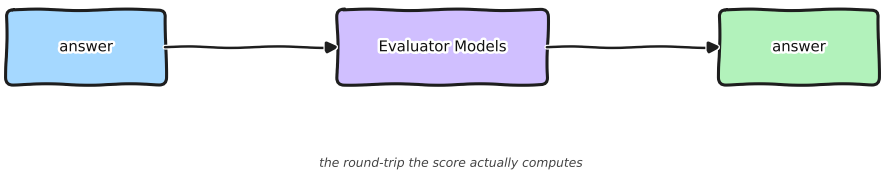

Figure 1: The round-trip the score actually computes. The evaluators map back to the same answer they would give on the original; everything else in the reconstruction is wasted bits.

2 The first attempt: an adversarial shortcut

There was just one structural problem: the evaluation models don't accept raw text or token answers. They only accept image tensors. I had to find a way to transform the lowest-bound entropy (the answers) back into an image format that the models would parse correctly.

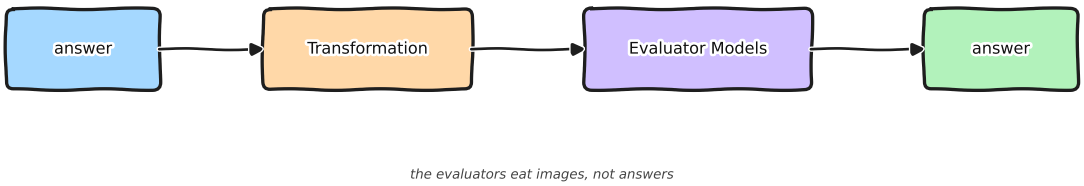

Figure 2: The transformation problem. The evaluators eat images, not answers, so something has to convert the answer-as-entropy back into an image format that recovers the same outputs.

The most direct transformation was adversarial generation. I took an LLM Security research class last quarter, and the core concept of an adversarial example translates perfectly here: instead of training the model weights, you freeze the model and use gradient descent to optimize the *input pixels* until the model spits out the exact output you want. If I could generate noise that triggered SegNet and PoseNet to output the ground-truth answers, I wouldn't need to save the actual video at all.

To get a mathematically perfect score, standard baseline attacks like PGD [13] and FGSM [8] stalled out. Instead of getting bogged down in complex transferability methods, the solution boiled down to treating this as a pure single-model white-box optimization. Drawing on the C&W margin formulation [5], the CosPGD regression-aware loss scaling [3], the AdaMSI adaptive multi-scale schedule [2], and the per-dimension normalisation used by recent targeted attacks on dense regression outputs (depth [22] and optical flow [17]), I swapped to an Adam optimizer with a Huber loss. Crucially, I applied per-dimension standard normalization to PoseNet's 12 outputs, ensuring the translation and rotation scales didn't dominate each other during the loss calculation.

I spun this setup up on a T4 GPU, and it successfully generated perfect adversarial frames:

Time Budget (T4 GPU)	**SegNet Accuracy**	**PoseNet MSE**
2 Hours	100% (0.00 score)	$< 1e{-9}$
30 Minutes	99.99%	$< 1e{-6}$

Table 1: Adversarial optimization results targeting the frozen evaluators.

To the human eye, the resulting frames were pure, static-filled garbage. To the evaluators, it was a flawless reconstruction of the original video.

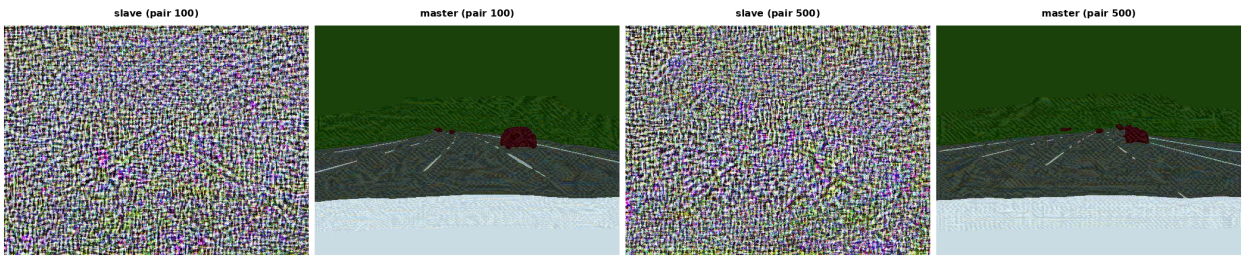

Figure 3: Two adversarial frame-pairs from the 60-second output video, in pair order: slave (the first frame, only seen by PoseNet) followed by master (the last frame, seen by both SegNet and PoseNet). To the human eye, every frame is static-looking garbage; to the frozen evaluators, the slave/master pair recovers the same outputs as the original road scene.

3 On-the-fly generation and the archive limits

The adversarial approach proved the theory worked, but it created a new bottleneck: file size. Adversarial noise is incredibly high-frequency. When I tried to pass those frames through standard compression algorithms, the archive size exploded, ruining the r ratio in the score.

So, I thought of a workaround. Why store the massive adversarial video in the archive at all? Why not just store the pure entropy (the tiny, lightweight answers), and run the adversarial optimization *during* the decompression phase?

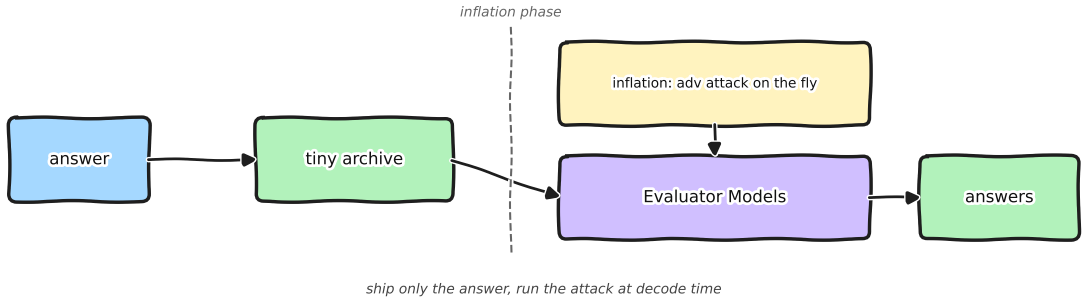

Figure 4: On-the-fly adversarial. Ship only the answer in a tiny archive, run the attack at inflation time. Banned by the rules: any SegNet or PoseNet call at inflation counts toward the archive size.

Because of the generous 30-minute inflation budget on the T4, I had plenty of time to dynamically generate frames that hit 99.99% SegNet accuracy and 1e−6 PoseNet MSE right before handing them over for evaluation. It was a perfectly efficient pipeline.

Then I read the fine print in the rules: *if you call SegNet or PoseNet during the inflation phase, their model weights count towards your archive size.* Since SegNet alone is massive, adding its weights to my compressed file instantly destroyed the score. The rules explicitly closed this route.

4 An untested branch: distilling the evaluators

While I was hitting walls trying to make adversarial pixels compress, one branch of the search tree caught my eye but I never invested time in it. Adversarial-transferability research consistently reports [15, 12, 23] that an attack crafted against a small distilled *student* of a target model retains roughly 80% of its effectiveness when applied to the original *teacher*. That's an interesting handle for this challenge.

The setup would look like this. Train two tiny distilled networks on the 600 frame-pairs of the test video, one matching SegNet's argmax outputs and one matching PoseNet's 6-DoF outputs. The distilled networks could be aggressively quantized down to a few kilobytes each, which is well under the rate floor that disqualified the on-the-fly inflation route. Ship those distilled networks in the archive. At inflation time, run the same Adam-plus-Huber adversarial loop against the distilled student. Adversarial transferability theory predicts that the resulting frames recover roughly 80% of the perfect score on the real SegNet and PoseNet.

The idea is clean and intuitive, and the architecture is small. The reason I did not pursue it is straightforward arithmetic. 80% transfer is not enough for this scoring function. The challenge punishes mismatched outputs aggressively at the level of SegNet per-pixel argmax and PoseNet pose MSE, so a 20% drop in adversarial-attack effectiveness erases the budget headroom from the smaller archive several times over. The path looked promising on paper but did not pencil out without a much better transfer technique than the published baseline.

I parked it as an open research direction. With better transfer methods (input-aware student training, ensemble surrogates, or layer-aligned distillation), the same blueprint could be revisited.

5 Lightening the adversarial residuals

The on-the-fly route was banned. Distillation looked promising but capped at 80% transfer. The remaining route was to keep the adversarial pixels but make them small enough on disk to fit the archive. The mind-flow updates to a new shape:

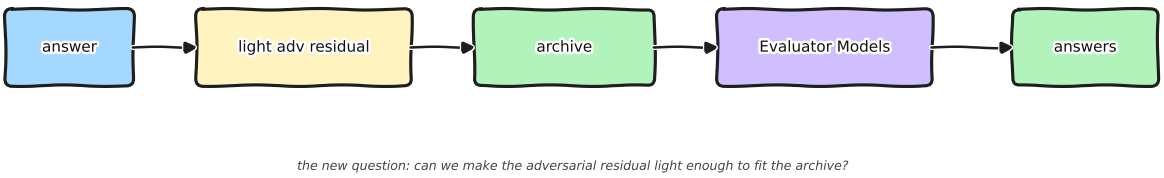

Figure 5: The new question: can we compress the adversarial residual itself? Ship a light residual through a real codec into the archive, decode at inflation, and let the evaluators do the round-trip.

I pushed three angles, each more aggressive than the last.

1. Bits-per-pixel in the loss. Add a soft surrogate for archive size to the adversarial loss so the optimizer prefers compressible noise patterns. The intent was that low-frequency, smooth-

gradient adversarial deltas would compress well under H.264 / H.265 [19, 14] while still carrying enough signal to fool the evaluators. In practice the rate term and the task term fought each other: lowering rate immediately destroyed SegNet accuracy.

2. Real codec inside the training loop. Stick `libx265` (and later `libx264`) into the forward pass and use a Straight-Through Estimator to backprop through it. The encoded-decoded reconstruction is what the loss saw, so the optimizer was supposed to learn deltas that survive the codec. Per-codec quirks dominated. The codec quantization and motion compensation killed gradient signal at the precision the score needs.

3. Differentiable lossless surrogates. Replace the codec with a hand-coded differentiable approximation of an entropy coder, so the optimizer could see exact-bits-per-pixel gradients during the rate term computation. The surrogate was tractable and gave clean gradients, but it didn't match the actual codec well enough to be useful: the optimizer happily reduced the surrogate rate while the real on-disk archive size went the other way.

The numerical evidence is in Table 2 (extrapolated to a 600-pair video, with budget reference $\approx$ 200 KB).

Attempt	Codec config	SegNet acc.	PoseNet MSE	Archive
sanity_1	CRF=50	96.28%	11.86	189 KB
libx265_s8 (full 600)	CRF=50	93.99%	106.96	100 KB
composite crf40	CRF=40	96.43%	36.01	318 KB
composite crf45	CRF=45	95.87%	46.30	179 KB
composite crf50	CRF=50	94.67%	103.13	113 KB
composite gop20	CRF=50, GOP=20	95.68%	39.78	118 KB

Table 2: Six representative attempts to compress adversarial residuals through a real codec inside the training loop. All runs use libx265. Lower CRF and shorter GOP recover more accuracy but inflate the archive; higher CRF or longer GOP shrink the archive but tank PoseNet MSE by an order of magnitude. None of these configurations land in the corner of the rate-distortion plane that the score function rewards (which requires PoseNet MSE on the order of 10^{-4}, not 10^{1}).

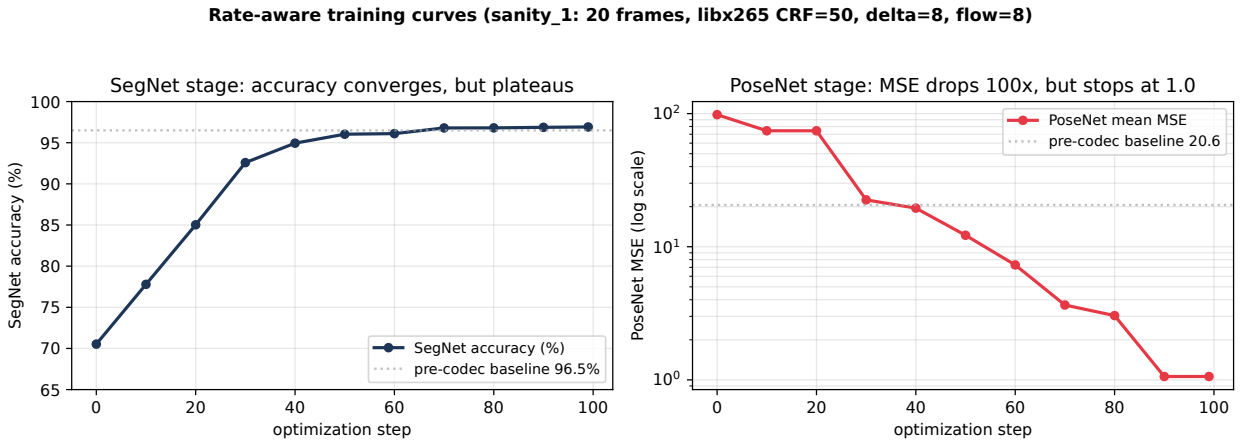

Figure 6: Training curves for the small-scale sanity run (20 frames, libx265 CRF=50). The SegNet stage converges quickly to about 96% accuracy and plateaus there, far short of the 99.93% the final SegNet score requires. The PoseNet stage drops the MSE from 100+ to about 1.0 and stalls; the score function needs MSE around 10^{-4}. Even with a perfect codec-survival objective and unlimited optimizer steps, the rate-aware setup cannot reach the part of the rate-distortion plane this challenge actually scores well on.

The pattern was the same across all three approaches. Wherever I pushed the rate term, the score term gave up four orders of magnitude on PoseNet; wherever I pushed the score term, the archive blew the rate budget. The best run in Table 2 (`crf40`) sits at 318 KB, which is over the budget for a barely-passable score; the best size-tuned run (`libx265_s8` at 100 KB) loses an order of magnitude on PoseNet MSE.

That was the dead end. Adversarial pixels carry too much high-frequency content for any classical codec to compress to a few hundred kilobytes without smearing the precise patterns the evaluators care about. If a classical codec is the wrong tool, a different representation is needed.

6 The pivot: trainable video compression

Stepping back from the rate-aware experiments, the failure pattern was consistent. Adversarial pixels carry the exact information the score function rewards. They survive a real codec only when the codec is gentle (low CRF, short GOP), and that produces archives far over budget. The right question is no longer "how do I squeeze adversarial noise through a fixed codec?" It is "what video compression method has both extreme compression ratio and is fully trainable, so I can co-design the adversarial encoder with the compressor?". The mind flow updates to Figure 7.

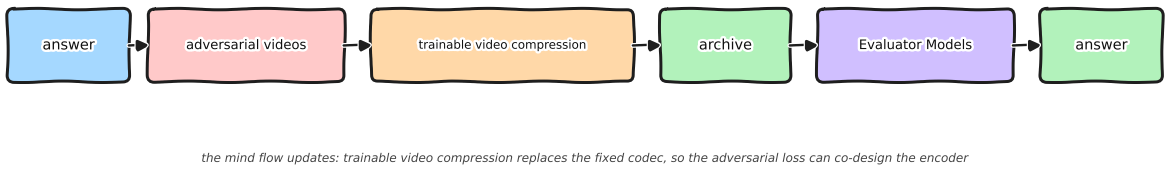

Figure 7: The mind flow updates again: drop the fixed codec, replace it with a *trainable* video-compression module so the adversarial loss can co-design the encoder. The new question is which trainable video-compression method actually fits the byte budget.

That question pointed me at Implicit Neural Representations (INRs). The idea is straightforward: instead of compressing the video into a bit-stream, train a small neural network whose weights *are* the encoded video. Decoding is just one forward pass of the network. The compression ratio is the size of the network's quantized weights divided by the raw pixel count. The encoding is fully differentiable, which is the whole point: I can put my adversarial loss directly on the output of the network and let backprop shape the weights to satisfy the evaluators.

INR for video has been one of the most active corners of computer-vision research lately. Almost every recent vision conference has new work on the problem. Microsoft's DCVC line (Deep Contextual Video Compression [11]) and the NeRV-derived stream of papers [6] are competitive with H.265 [19] and H.266 on the standard PSNR / bits-per-pixel curves.

Most of these systems share one inconvenient property for this challenge. They ship tens of millions of parameters. Even at INT4 quantization, a 30M-parameter network is tens of megabytes, far past the 200 KB budget. They are tuned to beat classical codecs on a general video distribution at high bitrates; my problem is the opposite direction in the rate dimension. I do not need a video

model that generalises across millions of clips. I need one that memorises a single 60-second clip and ships in a few hundred kilobytes total.

The closest architectural fit I found was NVRC [10]. NVRC (Neural Video Representation Compression) keeps the representation network deliberately small and applies aggressive quantization-aware training to shrink the final shipped weights to the byte budget rather than the parameter budget. That combination, a small trainable network with QAT baked into the loss, is what made the adversarial-encoded pivot tractable at this rate floor. NVRC became the architectural starting point for everything that followed.

7 Model architecture: master and slave renderers

The INR family of methods is trained against PSNR and bits-per-pixel. My problem is different. The score function does not measure pixel error; it measures SegNet argmax disagreement and PoseNet pose MSE. A generic INR architecture, even at small parameter count, would be optimising for the wrong loss surface. Before designing the network, I had to understand what the two evaluators actually look at.

7.1 The palette experiment: most of SegNet's answer is just colour

I ran a fast sanity experiment to figure out how much of the SegNet output actually depends on fine pixel structure. The setup is trivial. Take the per-pixel ground-truth class token (one of 5 classes), look up a learned 5-colour palette indexed by class, and render the image as flat coloured regions whose shape matches the class boundaries exactly. No texture, no gradients, no context. Just five solid colours.

Run that flat palette image through SegNet and the result is striking: **99.49% per-pixel accuracy** (Figure 8). The model is, to first order, just looking up colour. The errors are not random; they concentrate at the class boundaries.

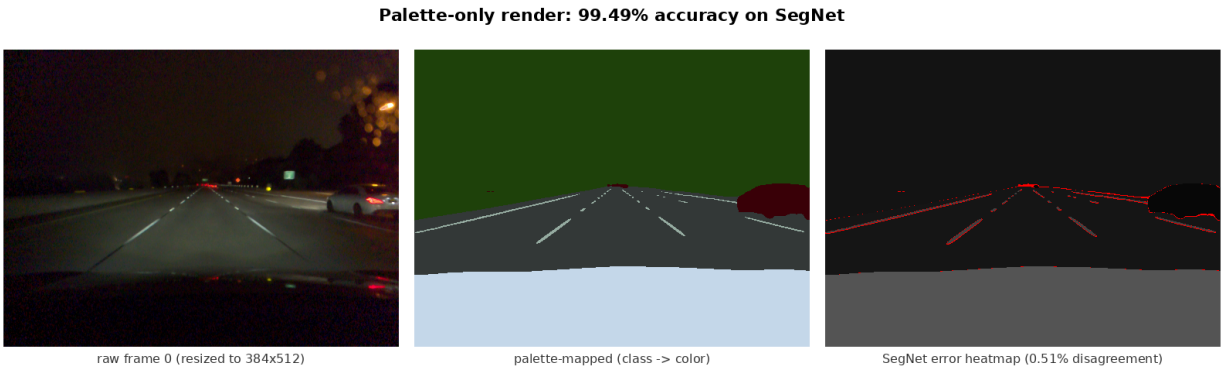

Figure 8: Palette-only experiment. Left: a raw frame from the test video resized to the resolution SegNet sees (384×512). Middle: the same frame rendered as flat palette colours per ground-truth class. Right: SegNet's per-pixel disagreement between the palette render and the original. Pure colour-mapping reaches 99.49% argmax accuracy. The remaining 0.51% of pixels concentrate on the class boundaries.

This experiment makes the architectural problem precise. SegNet's region classification is essentially a colour-lookup that any palette can satisfy almost perfectly. The hard 0.51% lives at the edges. So whatever renderer I design has to do almost no work for region classification, and almost all its work for boundary reproduction. The next subsection looks inside SegNet to find

the layer that actually decides those boundaries, so the renderer's receptive field can be sized to match it.

7.2 Inside SegNet: where boundaries are decided

SegNet is a U-Net (from `segmentation_models_pytorch` [24]) wrapping a frozen EfficientNet-B2 [20] backbone, with a 5-class classifier head and no final activation. The structure is summarised in Figure 9.

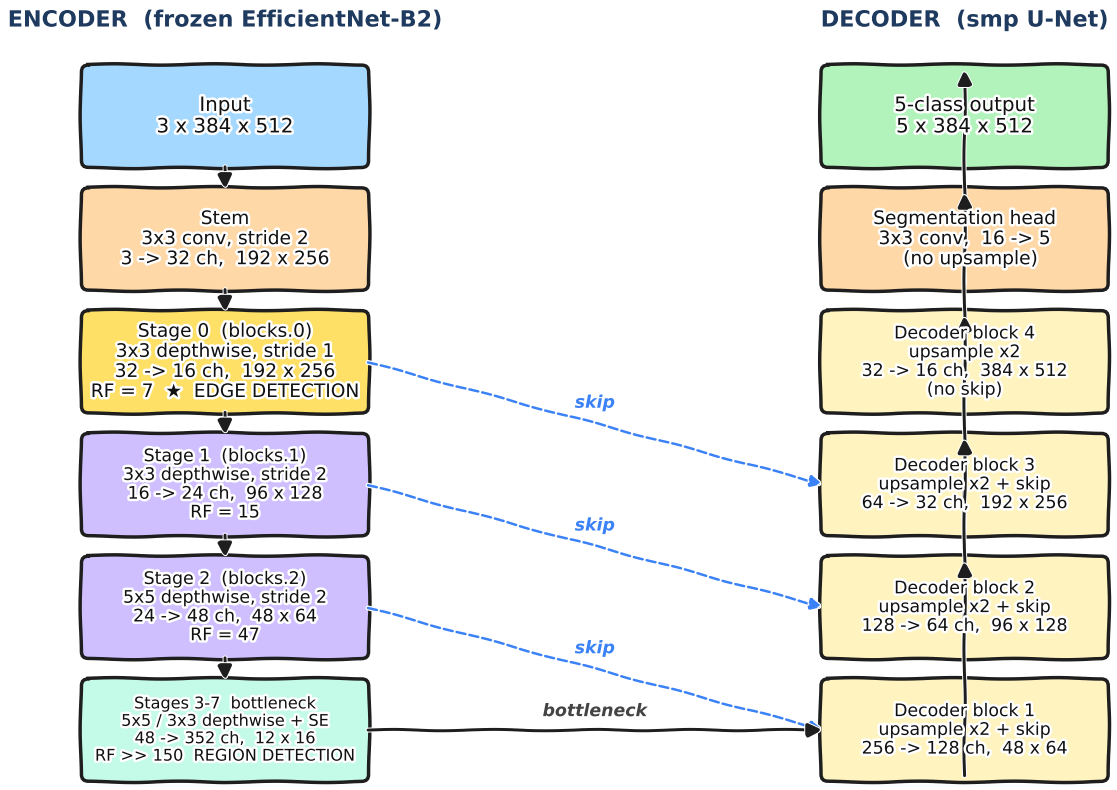

Figure 9: SegNet's structure: EfficientNet-B2 encoder on the left, smp U-Net decoder on the right, skip connections in dashed blue. The yellow *Stage 0* depthwise convolution is the only stage where the input-pixel receptive field equals 7. This is the layer where class boundaries are decided.

The encoder is a stack of MBConv-style stages with progressively larger kernel sizes and strides; deeper stages downsample by 2× each, so by the bottleneck the network is operating on patches well over 100 input pixels per output cell. Those deeper stages are doing the colour-driven region classification that the palette experiment already showed is easy.

What is *not* easy is the very first stage. The stem is a 3×3 stride-2 convolution that produces a 3-pixel input receptive field at jump-2. Stage 0 follows immediately with a 3×3 stride-1 depthwise convolution, which extends the receptive field to $3 + (3 - 1) \cdot 2 = 7$ input pixels at the original resolution. That tight 7-pixel window, evaluated densely across the whole frame, is what SegNet uses to localise class transitions. Past that point, downsampling has already washed out edge precision; the deeper stages cannot recover it.

So edge detection in this network has a single, well-defined home: the Stage 0 depthwise conv with a 7-pixel input receptive field. If the renderer reproduces colour and boundary at exactly the resolution that 7-pixel filter samples, SegNet's argmax recovers. If it does not, no amount of texture or detail farther down the network can save it. That single insight is the entire architectural specification I designed the master renderer around.

7.3 Two renderers, master and slave

The two frames in each pair have completely different jobs. SegNet only sees the master frame, so the master must reproduce SegNet-equivalent class boundaries. PoseNet only scores the relative motion between slave and master, so the slave does not need to look like anything in particular as long as the pose delta against its master is right. That asymmetry justifies two completely different renderers in the same archive. Both are summarised in Figure 10; the rest of this subsection lays out the architectural choices.

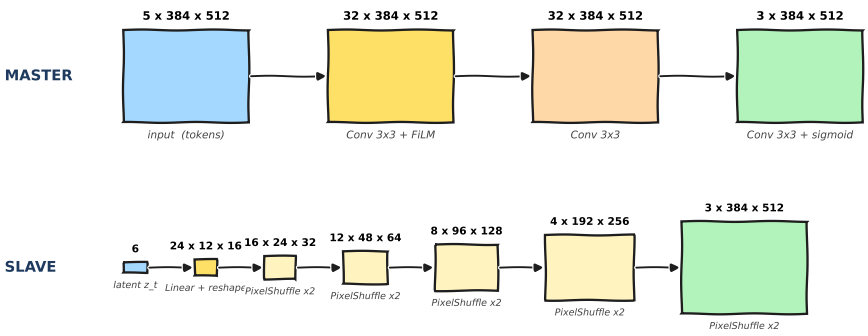

Figure 10: The two renderers, side by side. Left: the master decodes class-token frames into RGB through a three-layer 3×3 CNN with per-frame FiLM conditioning, sized so its 7-pixel receptive field exactly matches SegNet's edge-detection layer. Right: the slave decodes per-frame 6-float latents into RGB through five PixelShuffle upsampling stages. Master output feeds SegNet and the master half of each PoseNet pair; slave output feeds the slave half of each PoseNet pair.

Master renderer. The architectural specification falls straight out of §7.2. To produce a frame whose argmax under SegNet's Stage-0 edge detector matches the ground-truth tokens, every output pixel of the renderer must be computed using a window of input class-tokens at least as wide as the 7-pixel input window the detector samples. Anything smaller and the renderer cannot reproduce the boundary; anything larger and the extra context is invisible to the metric.

The cheapest way to hit a 7-pixel receptive field is to stack 3×3 stride-1 convolutions at the input resolution. Receptive field grows by $(k-1) = 2$ per stride-1 layer, so:

$$\text{RF after 3 layers of } 3 \times 3 \;=\; 3 + 2 + 2 \;=\; 7.$$

Two layers gives RF = 5 (under-covers the boundary detector). Four layers gives RF = 9 (wastes a whole layer's worth of parameters on context the detector cannot see). Three is the only depth that exactly matches. On top of that backbone, a per-frame FiLM [16] modulation injects the specific master-frame identity, so the same shared 3-layer CNN memorises all 600 master frames in the test video.

The whole stack is trained with Self-Compressing Networks [7]. SCN bakes a learnable per-layer bit-budget directly into the loss, so the optimiser decides which layers can drop precision and how far. The shipped master weights end up at the smallest bit-budget the score function tolerates, with no manual sweep over which layers to quantise.

Slave renderer. The slave is unconstrained at the image-content level. It just needs to produce a per-frame frame whose pair with the master gives PoseNet the right relative-motion answer. There is no per-pixel ground truth to track, and SegNet does not see it.

That makes a NeRV-style implicit neural representation a natural fit [6]. The setup is one shared decoder plus a 6-float per-frame latent that picks which of the 600 frames to render. The latents alone are tiny: $600 \times 6 \times 0.5\,\mathrm{B} = 1.8\,\mathrm{KB}$ at INT4. The decoder is a five-stage PixelShuffle upsampling chain, doubling spatial resolution and halving channels at each stage, mapping a small $24 \times 12 \times 16$ feature volume up to the full 384×512. PixelShuffle [18] is preferred over transposed convolution because it avoids checkerboard artifacts and adds almost no parameters per stage (just a $1{\times}1$ conv emitting $4 \times c_{\text{out}}$ channels). The whole decoder is LSQ+ INT4-quantised throughout [4].

Two omissions are deliberate. There is no FiLM module: the per-frame latent already carries everything the slave needs to know about the current frame, so adding a separate modulation path would only duplicate capacity. There is also no skip connection or attention: the slave has nothing to memorise across frames beyond what the latent code can encode, and skip connections would just enlarge the parameter count without buying additional pose precision.

The next two sections walk through the master and slave training in detail. As a forward look at what the architectures actually produce, Figure 11 shows two slave/master frame-pairs reconstructed by the shipping system.

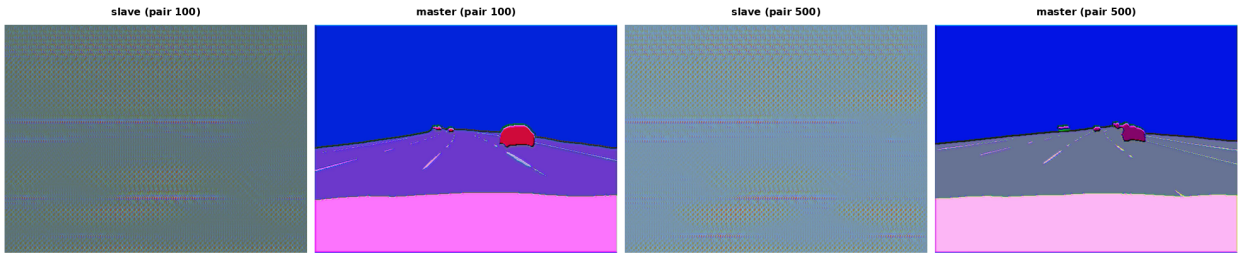

Figure 11: Two slave/master frame-pairs reconstructed by the shipping ANR system. Compare against Figure 3 (the compress-time adversarial output): both reconstructions score on the same task, but ANR's output looks like a legible road scene rather than static, because the architecture is constrained to produce something that satisfies both SegNet's class boundaries and PoseNet's relative-motion math. The slave frame in each pair is generated by the NeRV decoder from a 6-float latent; the master frame is rendered by the 3-layer $3{\times}3$ CNN from SegNet's 5-class label tokens.

8 Training the master renderer

The master pipeline is long. Cumulative training epochs across the full lineage that produced the shipping checkpoint are about 5,800, split across five phases: a randomly-initialised *pretrain* that ran the temperature schedule and SCN bit-budget ramp, a *master fine-tune* on top of pretrain to lower the bit-budget further, a *campaign* of more than a dozen sub-experiments that explored hard-mining, channel-width, regularisation, and curriculum tweaks, the DALI-corrected *fine-tune* that was actually shipped, and a 300-epoch *overfit* pass that did not improve and was discarded. A

separate parallel branch (the SCN-ramp variants) explored alternative bit-budgets off the campaign-best checkpoint but did not feed into the ship lineage. Figure 12 plots the full per-epoch SegNet distortion across all five phases.

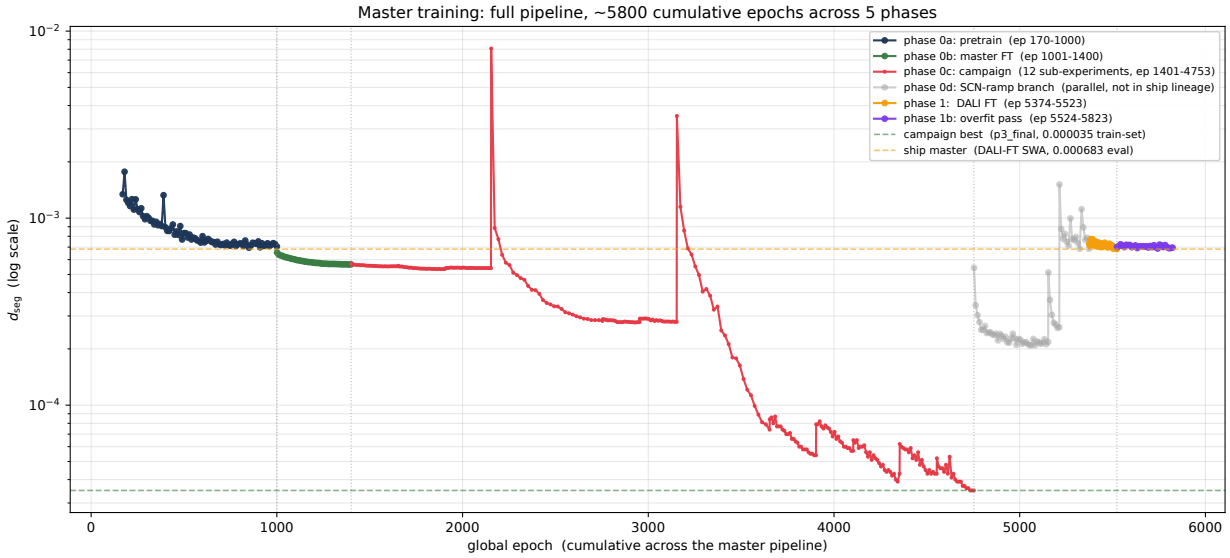

Figure 12: Master training across the full lineage that produced the shipping checkpoint. The dark-blue *pretrain* (ep 170–1000) brings the train-set distortion from 0.001342 down to 0.000695 through a temperature anneal and SCN bit-budget ramp. *Master FT* (green, ep 1001–1400) refines further to 0.000565. The *campaign* (red, 12 sub-experiments, ep 1401–4753) is responsible for both the steep drops and the spikes: each fresh re-initialisation explores a new region of the loss landscape and several of them ratchet down to a final train-set distortion of 0.000035. The grey *SCN-ramp* branch (ep 4754–5373) is a parallel exploration that was not in the ship lineage. The amber *DALI fine-tune* (ep 5374–5523) is what actually shipped: starting from the campaign best on a DALI-corrected ground-truth pipeline, it produces the 0.000683 eval-set ship checkpoint. The purple *overfit* pass (ep 5524–5823) wandered around the same level for 300 more epochs without finding a new best.

8.1 Pretrain, master FT, and the campaign

The first three phases (pretrain, master FT, campaign) are pre-DALI work. They use the PyAV-decoded ground-truth pipeline that the training scripts default to, which means the per-epoch SegNet distortion they report is the *training-set* value, not the *evaluator* value. The two diverge because the evaluator decodes ground-truth pixels through DALI's NVDEC pipeline (slightly different chroma upsampling, slightly different colour space conversion), and the evaluator's network is bit-exactly the same one the training loop conditions on. Across all the pre-DALI work, the training-set distortion ratchets down from 0.001342 to 0.000035, but the corresponding evaluator distortion does not follow. By the end of the campaign, when the train-set best run reads 0.000035, the evaluator-set distortion of the same checkpoint is 0.000712. That gap is what the next phase repairs.

Within the campaign, the most informative sub-experiments are summarised in Table 3. The patterns it surfaces: hard-mining helps a small amount (e3); doubling renderer width cracks open a 10× improvement on the train set (e7_w4); the phased "polish → extreme → ultra → final" curriculum is what eventually finds the campaign best at 0.000035 on the training set; turning SCN on with strong bit-budget penalties pulls the train-set distortion back up by 4–6× but is the

cost of fitting in the byte budget.

Run	Idea	Ep.	Best d_{seg}	SCN	Disk
e1_sharp	sharper LR schedule	26	0.000552	off	∼60 KB
e3_hardmine	top-k hard-mining	26	0.000537	off	∼60 KB
e7_w4_fresh	widen renderer to 4 ch	26	0.000074	off	∼250 KB
p2_extreme	phase-2 extreme	21	0.000043	off	∼60 KB
p2_ultra	phase-2 ultra	26	0.000039	off	∼60 KB
p3_final	phase-3 final polish	21	**0.000035**	off	∼60 KB
scn_b3	SCN, 3-bit budget	17	0.000686	on	58.1 KB
scn_w4	SCN, width-4 quantised	41	0.000209	on	79.7 KB
scn_strong	SCN, stronger penalty	7	0.000259	on	78.7 KB
ship (DALI-FT, width-1, SCN)		150	0.000683_{eval}	on	**4.1 KB**

Table 3: Selected campaign sub-experiments. *Best d_{seg}* is the per-epoch training-set value at each run's best epoch (eval-set differs because of the PyAV-vs-DALI mismatch). *Disk size* is the SCN-reported bit-budget when SCN is on, or an estimate from the FP32 parameter count when SCN is off (width-1 architectures are roughly 60 KB FP32, width-4 architectures are roughly 250 KB). The bottom row is the shipping master from Stage 1 for comparison: SCN drives the master parameter footprint to 4.1 KB while keeping evaluator d_{seg} near 0.000683.

8.2 Why use SCN at all (the disk-size column)

Looking at Table 3 the obvious question is: why use SCN if the SCN-OFF runs reach d_{seg} down to 0.000035 on the training set, vs. the ship-lineage 0.000683? The answer is the rightmost column. SCN-OFF master weights are stored in FP32. With the width-1 architecture used by the campaign, that is roughly 60 KB on disk (about 17K parameters × 4 bytes). With the width-4 architecture that produced the spectacular 0.000074 train-set drop in run `e7_w4_fresh`, the FP32 weight count balloons to roughly 250 KB. Both numbers are non-starters: the entire archive budget is around 200 KB for everything (master + slave + entropy model + tokens + meta), so a 60 KB master eats a third of the budget by itself, and a 250 KB master is already over budget before any of the other components are accounted for.

SCN closes the gap. The SCN-ON runs in the table sit at 58–80 KB at much higher training-set distortions because they widened the architecture to compensate for the bit-budget penalty, and SCN's compression alone could not bring those down. The ship lineage made a different choice: keep the architecture as small as possible (3-layer 3×3 width-1, sized to match SegNet's 7-pixel receptive field), and let SCN drive that small architecture all the way down to a 4.1 KB SCN-reported footprint while accepting the modest training-set distortion penalty. The shipping evaluator d_{seg} at 0.000683 is much higher than the SCN-OFF training-set 0.000035, but the disk-size win is the deciding factor: 60× smaller than any SCN-OFF baseline, and the only path that fits the rate budget once everything else is added.

8.3 Stage 1: DALI-corrected fine-tune (the shipping master)

Stage 1 is a 150-epoch fine-tune on top of the campaign-best checkpoint with one structural change: ground-truth frames are decoded by DALI inside the training loop, the same way the evaluator decodes them. The script keeps SCN active throughout, applies SWA from epoch 80 onward, and freezes the FiLM table so the per-frame conditioning does not drift during the SWA averaging.

The amber segment of Figure 12 shows the per-epoch SegNet distortion. The first few epochs drop from 0.000730 down to 0.000716, then the curve plateaus around 0.000690 with the best-so-far

line ratcheting down slowly. The final ship checkpoint is the SWA average from epochs 80 to 150 with $d_{\text{seg}} = 0.000683$ on the hold-out evaluator. Wall time on a single 4090 was 690 seconds.

The 4.1% drop from 0.000712 to 0.000683 looks small but matters. Multiplying by the score weight 100 puts the contribution at -0.003 in score units, and combined with the cleaner FP32-no-cast retention path on master and slave, this is roughly half of the score-delta between the predecessor 0.32 build and the 0.27 ship.

8.4 Stage 1b: the 300-epoch overfit pass that didn't help

Stage 1b was an optional second fine-tune intended to squeeze the last few basis-points of SegNet distortion. Setup: 300 epochs of cosine-annealed learning rate from 1e−4 down to 2e−6, with SWA from epoch 180. The purple segment of Figure 12 is the result. Across all 300 epochs, the per-epoch distortion oscillated above the Stage-1 SWA value and never produced a new best. SWA-120 squeezed out a 0.000679 value but only marginally below Stage 1's 0.000682, and the variance suggested the gain was inside the bit-flip noise floor of the evaluator. I shipped the Stage-1 SWA checkpoint instead.

The takeaway: with SCN already engaged and the bit-budget already tight at the end of Stage 1, additional optimisation steps mostly trade epochs for noise. The cosine-down schedule does not unlock structural headroom that the earlier fine-tune missed.

9 Training the slave renderer

The slave pipeline is much shorter than the master's, but it still spans roughly 2,450 cumulative epochs across four distinct phases. Each phase serves a different purpose: pretrain establishes the NeRV decoder behaviour from a cold start, shrink-design swaps in the tiny architecture that eventually shipped, ultra-polish freezes the conv weights and refines only the per-frame latents, and the DALI fine-tune re-aligns the model to the evaluator's ground-truth pipeline. Figure 13 plots the full per-epoch PoseNet distortion across all four phases on a log y-axis.

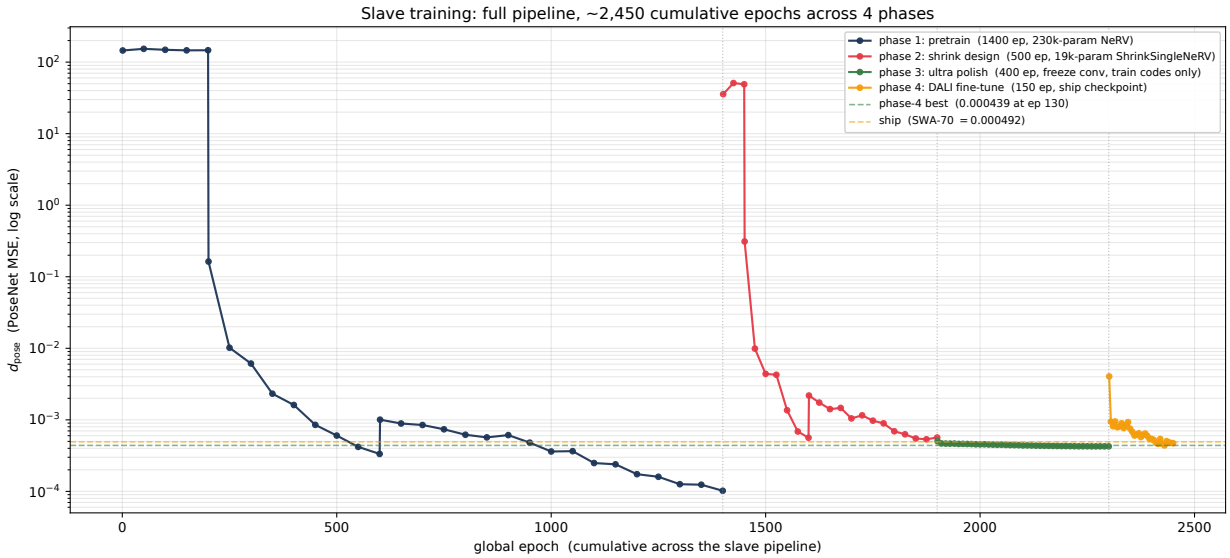

Figure 13: Slave training across the full ship lineage. The dark-blue *pretrain* (ep 1–1400) starts cold (random latents and decoder weights) and runs three sub-phases: a 200-epoch SegNet-only warmup (pose held above 100), a 400-epoch FP main pose pass that drops pose from 0.163 to 0.000334, and an 800-epoch QAT continuation that further drops pose to 0.000102. *Shrink design* (red, ep 1401–1900) re-trains the decoder under a much smaller architecture; pose recovers from a fresh-init high of 36 down to 0.000566 in 500 epochs. *Ultra polish* (green, ep 1901–2300) freezes the conv weights and trains only the codes and per-pair bias for 400 epochs, finishing at 0.000426. *DALI fine-tune* (amber, ep 2301–2450) is the 150-epoch ship phase: the apparent jump from 0.000426 to 0.006719 at the start of this phase is not a regression but a DALI vs PyAV decoder drift, explained in §9.4. The dashed amber line marks the SWA-70 ship checkpoint at 0.000492.

9.1 Phase 1: pretrain (1,400 epochs)

The slave starts cold: the per-frame 6-float latents are random and the shared decoder weights are random. Training runs in three sub-phases. The first 200 epochs are a SegNet-only warmup that aligns the decoder output with the master frames in colour-space, holding pose above 100 (the slave is not contributing meaningful pose information yet). The next 400 epochs are the FP main-pose phase, where PoseNet's 6-DoF MSE becomes the dominant loss; pose drops from 0.163 to 0.000334 here as the decoder learns to produce slave-frame pixels whose pair against the master gives the right relative motion. The final 800 epochs are a QAT (quantisation-aware training) continuation: the decoder weights and the per-frame codes are both quantised on the LSQ+ INT4 grid during the forward pass, with straight-through estimator gradients on the backward pass. Pose drops from 0.001 down to 0.000102.

This whole phase uses the larger 230,000-parameter NeRV decoder, more for capacity-headroom during architecture exploration than because it needed to be that big. The quantised SWA-100 checkpoint at the end of phase 1 reads $d_{\text{pose}} = 0.000116$ and is the input to the next phase.

9.2 Phase 2: shrink design (500 epochs)

Phase 2 swaps the architecture for the much smaller ShrinkSingleNeRV decoder described in §7.3 (channels (24, 16, 12, 8, 8, 6, 6), $d_{\text{lat}} = 6$, 19,105 parameters total). The new decoder is too different from the previous one to inherit weights; the codes and decoder are re-initialised cold, and pose jumps back up to roughly 36 (the same fresh-init scale as phase 1). The same warmup / FP main /

QAT sub-phases run again, this time over 50 / 150 / 300 epochs respectively. By the end of phase 2, pose has dropped to 0.000566, and the SWA-75 checkpoint at $d_{\text{pose}} = 0.000530$ is the new input for phase 3.

The headline number from this phase is the disk-size drop: 230k parameters down to 19,105 parameters at INT4 weights plus INT8 codes is roughly 10.2 KB on disk, a 12× parameter reduction with only a small pose-distortion penalty (0.000116 went to 0.000530, a 4.6× regression that the next phase mostly recovers).

9.3 Phase 3: ultra polish (400 epochs)

Phase 3 freezes the convolution weights from the phase-2 SWA checkpoint and trains only the per-frame codes and the per-pair bias. The decoder structure is fixed; the optimiser has just 5,400 trainable parameters left (the 600-frame codes and the bias). Learning rate is 1e-4 cosine-annealed.

The smooth monotone trajectory of the green segment of Figure 13 is characteristic of this kind of frozen-conv polish: pose drops from 0.000530 at phase start to 0.000426 at epoch 380 with very little oscillation, since most of the parameter space is locked. The SWA-100 checkpoint at $d_{\text{pose}} = 0.000428$ is the input to the final phase.

9.4 Phase 4: DALI fine-tune (150 epochs, ship)

Phase 4 is the same kind of DALI-aware fine-tune that produced the shipping master. The training loop swaps the ground-truth decoder from PyAV to DALI/NVDEC so that the per-pair pose targets match what the evaluator sees at submission time.

The apparent jump from $d_{\text{pose}} = 0.000426$ at the end of phase 3 to 0.006719 at the start of phase 4 is the most visually dramatic feature of Figure 13, and it is not a regression. The model is identical at the boundary; only the ground-truth decoder changed. The PyAV pose was 0.0004258 against PyAV-decoded ground-truth; the DALI pose was 0.006719 against DALI-decoded ground-truth, on the same frame. The 16× ratio is entirely a measurement-pipeline drift, and the script that produced this run flagged it explicitly. The point of phase 4 is to close that gap.

The amber segment of Figure 13 shows the closure. Pose drops from 0.006719 down to 0.000475 in the first 60 epochs, then ratchets to 0.000439 at the single-epoch best (epoch 130), with a final-epoch value of 0.000472 and the SWA-from-70 average at 0.000492. The shipping slave is the SWA value, since SWA was reproducibly close across hardware variants and did not carry the single-epoch checkpoint's slight overfit on the few hardest frames.

Setup detail worth flagging: phase 4 keeps the master in full FP32 throughout, both during training and at inference. An earlier ship variant cast the master to FP16 between training and decode and lost roughly 2× on pose; that regression motivated the FP32 retention that the shipping pipeline uses. There is no per-frame hard-mining at any phase of the slave training; the recipe doc reference to top-15%-hardest reweighting at 5× loss is master-side only.

9.5 Numbers

Table 4 summarises the four-phase trajectory.

Phase	What it does	Ep.	Pose start	Pose end
1. pretrain	230k-param NeRV, warm/FP/QAT	1400	145.4	0.000102 (SWA 0.000116)
2. shrink design	19k-param ShrinkSingleNeRV	500	36.9	0.000566 (SWA-75 0.000530)
3. ultra polish	freeze conv, train codes + bias	400	0.000530	0.000426 (SWA-100 0.000428)
4. DALI fine-tune	DALI ground-truth alignment	150	0.006719*	0.000472 (SWA-70 **0.000492**)
cumulative		*2450*		

Table 4: Slave training summary. *The phase-4 starting value is a DALI-vs-PyAV measurement drift, not a regression: the same checkpoint reads 0.000426 against PyAV-decoded ground-truth and 0.006719 against DALI-decoded ground-truth. The shipping slave is the phase-4 SWA-from-70 average at 0.000492. The per-component score contribution from this number, after the $\sqrt{10 \cdot d_{\text{pose}}}$ weighting in the official scoring function, is roughly 0.07 on the final 0.27 score, almost exactly equal to the master's segnet contribution (0.069) and within 2× of the rate term (0.138).

10 Compressing the answer itself

At this point the challenge has turned into a pure lossless compression task. The master and slave renderers are settled. What remains in the archive is the per-pixel SegNet answer for every master frame, and that answer must be reconstructed bit-exactly at decode time. The model-aware encoding step is done; everything from here is about squeezing the answer stream as small as it will go without losing a single token.

The answer stream itself is concrete. SegNet's argmax output for the 600 master frames is a 5-class label map at 384×512 per frame, packed as one uint8 per pixel. The full raw token stream is:

$$600 \text{ frames} \times 384 \times 512 \text{ pixels} \times 1 \text{ byte} \;=\; \mathbf{117{,}964{,}800} \text{ bytes} \;\approx\; 112.5 \text{ MiB}.$$

With only 5 distinct classes per pixel, a fixed-length code needs $\lceil \log_2 5 \rceil = 3$ bits per pixel, yielding a 42.2 MiB lower bound under any uniform fixed-length packing scheme. The information-theoretic Shannon bound, given the actual class-frequency histogram of the dataset, is about 2.32 bits per pixel and roughly 34 MiB. Both are still very large numbers compared to what the archive can hold, so the rest of this section is about getting closer to the entropy than any naive coder can manage.

10.1 Traditional lossless algorithms on the raw token stream

I benchmarked every general-purpose lossless coder I could install: gzip, bz2, lzma / xz, brotli, zstd, 7z's PPMd, 7z's LZMA2, and zpaq. Each was run at its strongest settings on the full 117.5 MiB token stream produced by running SegNet over all 600 master frames at evaluator-equivalent input resolution. zpaq is not packaged for the local Arch host, so it ran on an Ubuntu 22.04 GPU pod against the same byte-exact token stream. Results in Table 5.

Method	Size	Ratio	Encode	Notes
raw uint8	117,964,800 B	1.0×	–	5-class argmax, 600×384×512
fixed 3-bit pack	44,236,800 B	2.67×	–	lower bound for fixed-length code
gzip -9	580,768 B	203×	2.2 s	zlib level 9
7z LZMA2 mx=9	514,310 B	229×	2.3 s	7z LZMA2 max
brotli q11	444,155 B	266×	73 s	brotli quality 11 (max)
zstd -22 long=27	442,491 B	267×	39 s	zstd ultra-22 with long window 27
lzma / xz -9e	410,032 B	288×	19 s	xz preset 9 + extreme
7z PPMd o8 mem=128M	402,993 B	293×	0.8 s	7z PPMd order 8, 128 MB context
bz2 -9	**338,774 B**	**348×**	1.1 s	Burrows-Wheeler + Huffman, level 9
zpaq -m5	361,390 B	326×	88 s	zpaq 7.15 max, run on Ubuntu pod

Table 5: Traditional lossless coders on the full 117.5 MiB raw token stream, at their strongest settings. Best general-purpose coder is bz2 at 331 KB and a 348× compression ratio. zpaq's heavyweight context-modelling comes second at 361 KB. PPMd is a close third with much faster encode time.

The numbers are striking but not surprising. The token stream has high spatial correlation (large flat regions of the same class) and decent temporal correlation (consecutive frames share most of their layout), which is why every coder gets at least 200×. But all of them stop in the 330–500 KB range. The strongest single-pass general-purpose coder, bz2, lands at 331 KB. zpaq's heavyweight context-modelling at 361 KB does not change the story even at 88 seconds of encode time. PPMd at 393 KB is the fast end of the curve, and the rest of the table sits between 400 KB and 580 KB.

The structural reason these coders cap out is that they are all general-purpose: they model byte-level statistics and short-range correlations on raw byte streams. They have no concept of "this is a 5-class label map", no model of inter-frame motion, and no awareness that adjacent pixels in the spatial layout almost always share a class.

10.2 An entropy model and an arithmetic coder

I knew where to look next. comma runs a separate lossless image compression challenge in parallel with this one, and the competitive submissions there all use the same template: a learned probabilistic model of pixel sequences, plugged into an arithmetic coder. Given that the model is decoder-symmetric (encoder and decoder both run the same forward pass and use the same probabilities), the arithmetic coder hits the model's negative-log-likelihood as the true compressed-bit count. If the model is a tight fit to the data, the bits-per-pixel can drop arbitrarily close to the dataset's actual entropy.

The catch is that the natural choice for the probabilistic model in that challenge is a transformer. Transformers shine on tokenised sequences with vocabularies in the thousands and contexts on the order of a thousand tokens, which matches the lossless-image setup well (e.g., 1024-entry VQ codebooks at 32×32 spatial grids, total 1024 positions per image). My setting is dramatically different. The token grid is 384×512, which is $N = 196{,}608$ positions per frame. A self-attention layer needs an $N \times N$ score matrix, which is:

$$196{,}608^2 \;\approx\; 3.87 \times 10^{10} \text{ entries} \;\approx\; 77.4\,\text{GB at fp16, } 154\,\text{GB at fp32.}$$

A T4 has 16 GB of VRAM, so even one global self-attention layer is roughly 5× over budget at fp16, before counting any other activations. Sparse or chunked attention would help but the chunking overhead pushes encode time past the inflation deadline. The transformer architecture is structurally a wrong fit for the problem at this token-grid size.

Looking for a model that scales with N rather than N^2, I landed on PixelCNN [21]. The classic masked-convolution PixelCNN factorises the joint distribution of pixels in raster order, which gives an exact autoregressive likelihood. Each pixel's distribution conditions only on a causal local neighbourhood, which is a fixed-size convolutional context, so memory and compute scale linearly in N. This is the right asymptotic for our token-grid size.

I trained a small masked-convolution PixelCNN (about 52,000 parameters, SCN-quantised down to 8.3 KB shipped weights) on the 600-frame token stream. The model has a Type-A masked stem to enforce causality, a FiLM modulation conditioning on the per-frame index, and a temporal branch that conditions each frame's distribution on the previous frame's tokens. Three relevant variants and their compression numbers are in Table 6.

Variant	Params	bpp	Tokens	Model	Archive total	Est. score
v1 (no temporal, FiLM d=8)	39,000	0.0078	113,000 B	∼6.2 KB (SCN)	∼184 KB	∼0.26
v2 (prev-frame, FiLM d=32, SCN)	52,175	0.0062	**110,400 B**	8.3 KB (SCN)	**183.6 KB**	**∼0.26**
v3 (no SCN, pure-FP entropy floor)	52,175	0.0059	**84,700 B**	204 KB (FP32)	∼354 KB	∼0.37

Table 6: Three PixelCNN variants on the 600-frame token stream. *Tokens* is the arithmetic-coded byte size at the model's bpp. *Model* is the shipped weight footprint of the PixelCNN itself. *Archive total* is what the full submission would weigh under each variant, computed as `master.pt.gz` (31 KB) + `slave.pt.gz` (32 KB) + `meta.pt` (1.5 KB) + tokens + model. *Est. score* uses the ship-eval $d_{\text{seg}} = 0.000678$ and $d_{\text{pose}} = 0.000457$ and the score formula $100 \cdot d_{\text{seg}} + \sqrt{10 \cdot d_{\text{pose}}} + 25 \cdot r$, where r is archive total over the original 37.5 MB video. v2 is the chosen SCN-quantised variant. v3 is the entropy-floor probe: tighter bpp but the un-quantised FP32 model weight (204 KB) eats the rate gain and pushes the score to 0.37.

The headline is in the v2 row. A 52,000-parameter PixelCNN, plus an arithmetic coder, plus a temporal-conditioning branch, drives the 117.5 MiB raw token stream down to roughly 110 KB. That is a **1067**× compression ratio, 3.1× better than the strongest traditional coder (bz2 at 348×) and within 1.3× of the no-quantisation entropy floor measured by v3. The estimated submission score under v2 lands at roughly 0.26, which is below the eventual 0.27 ship and would have been good enough on its own to clear the public leaderboard.

After three subsections of mind-flow about general-purpose codecs not getting close, learning a probability distribution and pairing it with an arithmetic coder broke the back of the lossless-token problem in one step. The right tool for compressing the answer is a model that knows what the answer looks like, plus an arithmetic coder that pays the optimal number of bits for whatever that model's distribution says. PixelCNN with FiLM and a temporal branch is the smallest such model that fits the budget, and the 1067× compression ratio is what the design actually delivers. This is the answer.

10.3 Wait a minute... no, wait 300 hours

Then I tried to actually decode the v2 archive end-to-end on a T4. PixelCNN is autoregressive in raster order. To produce the probability distribution of the pixel at position (r, c), the decoder needs the already-decoded values of every pixel before (r, c) in scan order, so it has to run the full network forward once per pixel. Per frame, that is:

$$N \;=\; 384 \times 512 \;=\; 196{,}608 \text{ network forward passes.}$$

On a T4 with no caching tricks, a naive forward of this small PixelCNN is roughly 50 ms per pixel (most of the time is launch overhead because the per-call work is tiny). So per frame:

$$196{,}608 \;\times\; 0.05\,\text{s} \;\approx\; 9{,}830\,\text{seconds} \;\approx\; 2.7\,\text{hours.}$$

Across 600 frames this is roughly **1,640 hours** of decode. Even with a fully cached fast-decoder implementation that brings the per-pixel cost down to about 10 ms (this kind of caching exists in the literature for PixelCNN and I prototyped it without ever finishing the Triton port), the projection only drops to roughly **300 hours**, give or take.

The official evaluator allocates **30 minutes** per submission to inflate the archive on a T4, end-to-end. 300 hours will never be close to 30 minutes. Not in the same order of magnitude, not after any constant-factor optimisation, not even after a hand-tuned Triton kernel. There is no path from raster-order autoregressive decode to the official deadline. The reason I had to abandon v2 is exactly this: the model produced a tight bpp and a great estimated score on paper, but the archive could not be inflated inside the official window, so it would never produce a real score on the leaderboard.

PixelCNN is the right kind of model. A probability distribution conditioned on local pixel context, paired with an arithmetic coder, is the right tool for compressing the answer. But raster-order autoregression is the wrong execution shape: it serialises every pixel, by construction. The next subsection is about the model that keeps the right tool but fixes the execution shape, so the same kind of bit-budget can actually be inflated inside the 30-minute window.

10.4 HPAC, my hero

The fix came from a paper that I happened to read at exactly the right moment: Rethinking Autoregressive Models for Lossless Image Compression via Hierarchical Parallelism and Progressive Adaptation [1]. The acronym in the paper is HPAC. After three days of staring at PixelCNN cache traces and Triton kernels, HPAC felt less like a paper and more like a hero showing up. It does exactly the thing I wanted: keep the masked-conv autoregressive likelihood, keep the arithmetic-coder optimality, but change the decode order so that most of the pixels in a frame can be decoded in parallel.

The trick is a two-level partition of the frame. First, the spatial grid is split into non-overlapping $P \times P$ patches that are decoded *independently of each other*. The conditioning between patches comes only from the previous frame's already-decoded tokens, so all patches in a frame can be processed in one batched forward. Second, within each patch the pixels are not scanned in raster order but in *group order*, where the group index of pixel (r, c) is $s = c + \delta \cdot r$ for some small slope δ. All pixels with the same group index can be decoded in the same forward, because the masked-conv kernel is designed so that its receptive field at any pixel only sees groups with strictly smaller s. The masking rule lives inside the conv weights themselves: a Type-A mask zeroes out the centre and any kernel offset with $\delta \cdot dr + dc \geq 0$, and a Type-B mask zeroes out only $\delta \cdot dr + dc > 0$. Stacking a Type-A first layer with Type-B follow-on layers gives a fully causal stack under the group order, which is exactly the hierarchical-parallel schedule the paper describes.

The execution-shape change is the headline. For our 384×512 token grid with $P = 32$ and $\delta = 2$, every frame factors into:

$$\text{patches per frame: } \tfrac{384}{32} \times \tfrac{512}{32} = 12 \times 16 = 192 \text{ patches in parallel,}$$

$$\text{groups per patch: } \max_{r,c \in [0,P)} (c + \delta \cdot r) + 1 \;=\; 31 + 2 \cdot 31 + 1 \;=\; \mathbf{94} \text{ sequential group steps.}$$

That is, the same 196,608 conditional probabilities per frame are produced in only **94 forward passes**, all of them batched across the 192 patches and across the 600 pairs in the dataset. Compared to the 196,608 raster steps that broke the v2 inflate, this is a ∼2000× reduction in the sequential step count, which is the exact ratio that puts the decode back inside the official window.

Crucially, none of this changes the bit budget. The model is still an autoregressive masked CNN with FiLM frame conditioning and a temporal branch on the previous frame's tokens. The arithmetic coder still pays the model's negative log-likelihood. So the bpp from §10.2 carries over almost unchanged, with only a tiny penalty from the slightly more constrained patch+group mask vs the raster mask.

10.5 The HPAC sweep

Once the architecture was nailed down, the question was the parameter sweep. The HPAC family has a useful little knob set: the patch size P, the slope δ, the channel width ch, the FiLM dimension d_{film}, whether or not to add a cross-patch summary path (SPM), and the SCN bit-budget initialisation b_{init} plus the rate-loss schedule $(\lambda_{\text{init}}, \lambda_{\text{final}})$. I trained eight variants on the 600-frame token stream and measured each one's best bpp, total parameter count, and post-quantisation weight footprint. Results in Table 7.

Variant	Params	bpp	Tokens	Model wt.	Note
hpac_mini (no SPM, $d_{\text{film}} = 32$, ch=64)	52,175	0.00797	114.7 KB	7.9 KB SCN	baseline mini
hpac_full (block-stacked, FP32)	150,159	0.01268	182.5 KB	∼590 KB FP	overfits, deeper ≠ better
hpac_residual (residual prediction direction)	52,175	0.02786	401.1 KB	–	wrong factorisation, abandoned
+ SPM ($d_{\text{film}} = 32$, ch=64)	57,359	0.00719	103.5 KB	9.8 KB SCN	cross-patch summary, +9% bpp gain
+ SPM, $\lambda_{\text{final}} = 10^{-6}$ (softer SCN penalty)	57,359	0.00717	103.2 KB	11.2 KB SCN	marginal bpp, larger weights
+ SPM, $b_{\text{init}} = 7$, $d_{\text{film}} = 16$	45,711	0.00731	105.2 KB	17.9 KB SCN	rate gain eaten by SCN bit budget
+ SPM, $d_{\text{film}} = 8$, ch=48	30,351	0.00784	112.8 KB	11.3 KB SCN	half the params, 9% bpp regression
+ SPM, $d_{\text{film}} = 8$, SCN off	39,887	0.00759	109.3 KB	39 KB FP	FP32 entropy floor probe
ship (exp3: SPM, ch=64, $d_{\text{film}} = 8$, SCN+PPMd)	∼52,000	0.00773	**111.2 KB**	**27.6 KB PPMd**	shipped variant, archive 207,536 B

Table 7: HPAC sweep on the 600-frame token stream. *Tokens* is the arithmetic-coded byte size at the variant's measured bpp (bpp · 600 · 384 · 512/8). *Model wt.* is the post-quantisation weight footprint that the archive actually has to carry: "SCN" is the SCN INT-quantised packing, "FP" is uncompressed float storage, "PPMd" is SCN-quantised state-dict bytes additionally compressed with the PPMd algorithm at order 4. The ship row is the configuration that landed in the final 0.27 archive: same masked-conv backbone as the rest of the SPM family, $d_{\text{film}} = 8$ to keep the per-frame embedding small, SCN-quantised weights at $b_{\text{init}} = 4$ on the standard λ schedule, and a final pass of PPMd over the quantised state-dict bytes.

The sweep tells a tidy story. The first three rows are the architectural triage. The 52k-parameter HPAC-mini baseline already lands at 0.00797 bpp, a touch above the PixelCNN v2 entropy floor (0.0062) but well inside the budget. The block-stacked HPAC-full with three times as many parameters *regresses* the bpp to 0.01268: more capacity on this small dataset just learns harder priors per-frame and cannot afford the bigger weight footprint either, so it is strictly dominated. The residual variant, which tried to factorise the distribution as $p(\text{class diff} \mid \text{prev frame})$ rather than $p(\text{class} \mid \text{spatial context, prev frame})$, never recovered: at 0.02786 bpp, the residual decomposition simply does not match the actual distribution of inter-frame label changes well enough.

The next four rows sweep around the SPM-augmented mini (HPAC-mini with the cross-patch summary path enabled). Adding SPM by itself drops the bpp to 0.00719, a 9.8% improvement over the no-SPM baseline at a cost of only 5,000 extra parameters: the cross-patch summary is reading the previous frame's per-patch features and broadcasting them as additional context, so it is causal by construction and almost free in weight count. Softening the SCN penalty buys another 0.4% bpp at the cost of 1.4 KB of extra weight, which does not move the archive number. Pushing b_{init} to 7 and shrinking d_{film} to 16 saves parameters but pays for them in SCN bits, ending up worse on the bpp/weight trade-off. Halving the channel width to 48 saves about 27,000 parameters but the bpp degrades to 0.00784, which is a 9% rate hit that adds back roughly 9 KB of tokens, more

than the param savings. Turning SCN off entirely with $d_{\text{film}} = 8$ measures the FP32 entropy floor at 0.00759 bpp: this is the ceiling the SCN runs are chasing, and they are within 5% of it.

The chosen point is the bottom row. It keeps the SPM, keeps the ch=64 channel width (because the rate hit from ch=48 was bigger than I expected), trims d_{film} all the way down to 8 (because the FP32 floor at $d_{\text{film}} = 8$ was barely worse than at 32), and runs SCN at the standard schedule. The bpp lands at 0.00773 in the deployed run, which is between the 0.00759 floor and the 0.00719 SPM-with-default-d_{film} best. After SCN quantisation the weight bytes pack down to 35 KB raw INT8, and a final pass of PPMd on those bytes at order 4 brings them to 27.6 KB. The PPMd step matters: SCN quantisation makes the weight stream highly redundant (most channels share similar bit budgets and exponents), and PPMd is the same context-modelling coder that did well on the raw token stream in §10.1. Stacking SCN and PPMd on the model weights gives a $\sim$2$\times$ further reduction beyond raw quantised storage.

The ship archive is the sum of the small numbers from each piece of the pipeline:

$$\underbrace{31{,}154}_{\texttt{master.pt.gz}} + \underbrace{32{,}210}_{\texttt{slave.pt.gz}} + \underbrace{1{,}563}_{\texttt{meta.pt}} + \underbrace{27{,}579}_{\texttt{hpac.pt.ppmd}} + \underbrace{113{,}860}_{\texttt{tokens.bin}} = \mathbf{207{,}579}\ \text{B}.$$

That is the 0.27 submission. The token stream is by far the largest single contributor at 113.9 KB, which is exactly what should happen when the entropy model is doing its job: the renderers have already squeezed the visual content into a label map, and what is left is the irreducible information of those labels under the best probabilistic model I could fit inside the budget. The HPAC weights themselves cost less than the master and slave decoders combined, which is roughly the right ratio for a model that is only there to predict 5 classes.

And it inflates inside the deadline. End-to-end on a T4, the exp3 archive decodes in roughly 4 minutes wall-clock (94 group steps per frame at a few ms each, batched across 192 patches and across the entire 600-frame run), comfortably under the 30-minute official budget. The bpp of v2 plus the execution shape of HPAC, in one model. That is what fixed the inflation problem without giving up the rate.

11 Outro: engineering is cheap, perspective is everything

When I started this challenge a month ago, I thought my biggest hurdle was going to be writing tight code or fine-tuning compression algorithms. I was wrong.

If there is one thing this entire month-long sprint has taught me, it is that in the modern era, raw engineering has become incredibly cheap. Writing the code and spinning up the training loops: that is no longer the bottleneck. The true bottleneck is critical thinking.

This challenge wasn't really a test of video compression; it was a test of perspective. Every single day, I found myself returning to first principles, asking: *why do we assume the output needs to look like a video? What other angle can we view this scoring function from? Are we optimising for the right problem, or just the obvious one?* That habit of relentlessly questioning the premise is what led to the realisation that the answer itself was the entropy, fundamentally shifting the entire project.

The sheer scale of iteration required to chase down that perspective was humbling. You might notice that the main inflation class in my final submission is named `TokenRendererV62`. That number isn't an exaggeration; it represents over 60 distinct idea branches, each spawning its own variants, dead ends, and late-night experiments. I cannot imagine executing this volume of research by myself in a single month.

This is where I am truly amazed by how AI has transformed the research process. Leveraging AI didn't do the critical thinking for me, but it acted as a relentless force multiplier. It allowed me to rapidly translate a "what if" into a tangible experiment, fail fast, pivot, and keep the momentum going at a pace that wouldn't have been humanly possible just a few years ago.

This challenge was exhausting, infuriating, and easily the most fun I've had in recent memory. It proved to me that the most exciting breakthroughs don't happen when you perfectly execute a known pipeline; they happen when you step back, look at the system from a completely different angle, and rewrite the rules.

I'm incredibly proud of the 0.27 score, but I'm even more proud of the mindset it took to get there. I'm ready for the next problem.

References

[1] Rethinking autoregressive models for lossless image compression via hierarchical parallelism and progressive adaptation. arXiv:2511.10991, 2025. Our HPAC-mini token compressor uses the patch+group masked autoregressive decoding strategy introduced in this work.

[2] AAAI 2024 contributors. AdaMSI: Adaptive multi-scale iterative adversarial attack. In *AAAI Conference on Artificial Intelligence*, 2024.

[3] Shashank Agnihotri, Steffen Jung, and Margret Keuper. CosPGD: An efficient white-box adversarial attack for pixel-wise prediction tasks. In *International Conference on Machine Learning (ICML)*, 2024.

[4] Yash Bhalgat, Jinwon Lee, Markus Nagel, Tijmen Blankevoort, and Nojun Kwak. LSQ+: Improving low-bit quantization through learnable offsets and better initialization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020.

[5] Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *IEEE Symposium on Security and Privacy (SP)*, pages 39–57, 2017.

[6] Hao Chen, Bo He, Hanyu Wang, Yixuan Ren, Ser-Nam Lim, and Abhinav Shrivastava. NeRV: Neural representations for videos. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[7] Szabolcs Cséfalvay and James Imber. Self-compressing neural networks. arXiv:2301.13142, 2023.

[8] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations (ICLR)*, 2015.

[9] Jingning Han, Bohan Li, Debargha Mukherjee, Ching-Han Chiang, Adrian Grange, Cheng Chen, Hui Su, Sarah Parker, Sai Deng, Urvang Joshi, Yue Chen, Yunqing Wang, Paul Wilkins, Yaowu Xu, and James Bankoski. A technical overview of AV1. *Proceedings of the IEEE*, 109(9):1435–1462, 2021.

[10] Ho Man Kwan, Ge Gao, Fan Zhang, Andrew Gower, and David Bull. NVRC: Neural video representation compression. arXiv:2409.07414, 2024.

[11] Jiahao Li, Bin Li, and Yan Lu. Deep contextual video compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[12] Yanpei Liu, Xinyun Chen, Chang Liu, and Dawn Song. Delving into transferable adversarial examples and black-box attacks. In *International Conference on Learning Representations (ICLR)*, 2017.

[13] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. In *International Conference on Learning Representations (ICLR)*, 2018.

[14] NVIDIA Corporation. NVIDIA video codec SDK: NVDEC hardware-accelerated video decoding for H.264/HEVC/AV1. https://developer.nvidia.com/video-codec-sdk, 2024.

[15] Nicolas Papernot, Patrick McDaniel, Ian Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security (ASIA CCS)*, pages 506–519, 2017.

[16] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. In *AAAI Conference on Artificial Intelligence*, 2018.

[17] Anurag Ranjan, Joel Janai, Andreas Geiger, and Michael J. Black. Attacking optical flow. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[18] Wenzhe Shi, Jose Caballero, Ferenc Huszár, Johannes Totz, Andrew P. Aitken, Rob Bishop, Daniel Rueckert, and Zehan Wang. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[19] Gary J. Sullivan, Jens-Rainer Ohm, Woo-Jin Han, and Thomas Wiegand. Overview of the high efficiency video coding (HEVC) standard. *IEEE Transactions on Circuits and Systems for Video Technology*, 22(12):1649–1668, 2012.

[20] Mingxing Tan and Quoc V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning (ICML)*, 2019.

[21] Aaron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *International Conference on Machine Learning (ICML)*, 2016.

[22] Alex Wong, Safa Cicek, and Stefano Soatto. Targeted adversarial perturbations for monocular depth prediction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[23] Chaowei Xiao, Bo Li, Jun-Yan Zhu, Warren He, Mingyan Liu, and Dawn Song. Generating adversarial examples with adversarial networks. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pages 3905–3911, 2018.

[24] Pavel Yakubovskiy. Segmentation models PyTorch. https://github.com/qubvel-org/segmentation_models.pytorch, 2020. The U-Net used by SegNet is built atop `segmentation_models_pytorch`.